An

Internship Report on

Agentic Ai - HealthGuardian Ai

Submitted

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE-Artificial Intelligence and Machine Learning

By

T.Hansika

24R11A66R4



Department of CSE-Artificial Intelligence and Machine Learning

Geethanjali College of Engineering and Technology

(UGC AUTONOMOUS INSTITUTION)

(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

August -2026

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

Department of CSE-Artificial Intelligence and Machine Learning

Geethanjali

CERTIFICATE

This is to certify that the internship report titled **Agentic Ai-HealthGuardian Ai** being submitted by **T.Hansika,** bearing roll number **24R11A66R4,** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology "**CSE-Artificial Intelligence and Machine Learning**" from Jawaharlal Nehru Technological University, Hyderabad during the year 2026-2027.

Internship Supervisor
Mr.G.Raju
(Asst.Prof, CSE-AI&ML)

Internship Coordinator
Mr. Vedavyas Gurla
(Assoc.Prof CSE-AI&ML)

Dr. L. Venkateswarlu
(Professor & HoD, CSE-AI&ML)

Examiner
Signature:
Name:
Designation:



URVAH DYNAMICS
PRIVATE LIMITED

CERTIFICATE

OF INTERNSHIP

THIS CERTIFICATE IS PRESENTED TO

Hansika Togaru

has successfully completed the 30 days Agentic AI Internship Program organized by Urvah Dynamics Pvt. Ltd.
During the internship, the participant demonstrated dedication, enthusiasm, and a commitment to learning while gaining practical knowledge in Python, Data Analysis, Machine Learning, Deep Learning, Django, Git & GitHub, Power BI, and Agentic AI through hands-on projects and interactive sessions.

TATIPAMULA BALADITYA
Co-Founder & CTO

Geethanjali College of Engineering & Technology

(UGC Autonomous)
(Affiliated to JNTUH Approved by AICTE, New Delhi)
Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

Department of CSE-Artificial Intelligence and Machine Learning



Geethanjali

DECLARATION BY THE CANDIDATE

I, T.Hansika , bearing Roll No. 24R11A66R4 , hereby declare that the Internship Report entitled **"HealthGuardian-Ai"** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **CSE-Artificial Intelligence and Machine Learning**.

This is a record of bonafide work carried out by me in **Urvah Dynamics Pvt Ltd** and the results embodied in this internship have not been reproduced or copied from any source. The results embodied in this Internship Report have not been submitted to any other University or Institute.

Signature of Student
(T.Hansika,24R11A66R4)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who played a role in the successful completion of this project. My deepest thanks go to our Director, **Dr. Uday Kumar Susarla**, and our college Principal, **Dr. K. Sagar**, for providing the essential resources and unwavering support for this endeavor. I am profoundly grateful to the Professor & Dean - School of CS and Informatics **Dr. V. Madhusudan Rao** and our Head of the Department**. Dr. L. Venkateswarlu** for his invaluable guidance and encouragement throughout the internship project.

I would like to extend my heartfelt gratitude to our beloved faculty as Internship supervisor **Mr.G.Raju** whose invaluable expertise and guidance were instrumental to our success. I am also sincerely appreciative of our Internship Coordinator, **Mr.Vedavyas Gurla**, for the unwavering support and encouragement throughout this period. Additionally, I am deeply grateful to **Urvah Dynamics Pvt Ltd** and my internship guide **Mr.Balaji** for this incredible opportunity, continuous training, and invaluable guidance throughout my journey. Thank you all for your significant contributions to my learning and growth.

Introduction about Internship Organization

Urvah Dynamics Pvt. Ltd. is a technology-oriented organization focused on providing practical exposure and skill development in emerging areas of software technology and artificial intelligence. The organization emphasizes hands-on learning and encourages students to apply theoretical concepts to real-world problems through structured internship programs and project-based training.

The organization provides learning opportunities in areas such as Python programming, data analysis, machine learning, deep learning, web development, and Artificial Intelligence. Its internship programs are designed to help students understand modern technologies and develop practical skills through implementation, experimentation, and project development. The training also provides exposure to tools and technologies used in the software and AI development ecosystem, along with professional practices such as Git and GitHub.

As part of my 30-day internship at Urvah Dynamics Pvt. Ltd., I received practical training in Artificial Intelligence and Agentic AI. The internship introduced me to the concepts of intelligent systems, machine learning, generative AI, and AI agents, along with their applications in solving real-world problems. The training helped me understand how different AI technologies can be integrated to develop intelligent and user-oriented applications.

The knowledge and practical experience gained during the internship were applied to the development of **Health Guardian AI**, an AI-based healthcare application. The project combines machine learning with generative AI and intelligent-agent concepts to support health-related prediction and provide useful, personalized recommendations. Through this project, the concepts learned during the internship were transformed into a practical application addressing a real-world healthcare problem.

Overall, the internship at Urvah Dynamics Pvt. Ltd. provided valuable exposure to emerging AI technologies and practical software development. It helped bridge the gap between academic learning and industry-oriented application development while improving my technical, analytical, problem-solving, and project implementation skills.

Training Schedule

Week	Training Schedule
WEEK 1	**Introduction to Artificial Intelligence and Python:** Introduction to AI concepts, Python programming fundamentals, data handling using NumPy and Pandas, and understanding the role of AI in real-world applications.
WEEK 2	**Machine Learning:** Introduction to machine learning, supervised learning concepts, regression techniques, data preprocessing, model training, evaluation, and practical implementation using Python.
WEEK 3	**Generative AI and Agentic AI:** Introduction to Generative AI, large language models, prompt engineering, AI agents, agentic workflows, and understanding how intelligent systems can perform tasks using AI models.
WEEK 4	**Project Development – Health Guardian AI:** Integration of the concepts learned during the internship, development and testing of the Health Guardian AI application, implementation of machine learning and AI-based functionalities, debugging, and deployment using Streamlit.

TABLE OF CONTENTS

S. No	Contents	Page No
	Abstract	**i**
	List of Figures	**ii**
	List of Tables	**iii**
	List of Screenshots	**iv**
	List of Symbols & Abbreviations	**v**
1	**Introduction**	**1**
2	**Related work/ Technologies used**	**2..**
3	**Work done/ Observation/ Duties performed**	**3..**
4	**Learning after Internship**	
5	**Summary / Conclusion**	
6	**Bibliography**	

List of Figures/Diagrams/Graphs

List of Tables

List of Screenshots

List of Abbreviations

Important Note:

- **All the above pages are to be numbered in Roman numerals of lower case. Ex. i, ii, iii, iv,…**
- **The Table of contents must be numbered using numbers i.e. 1, 2, 3……**

ABSTRACT

Healthcare is an important area where Artificial Intelligence and Machine Learning can be applied to support early identification of potential health risks and provide accessible health-related guidance. During the internship at Urvah Dynamics Pvt. Ltd., practical knowledge was gained in Artificial Intelligence, Machine Learning, Generative AI, and Agentic AI. This knowledge was applied to develop **Health Guardian AI**, an intelligent healthcare application designed to provide health-related predictions and guidance based on user-provided information.

Health Guardian AI uses machine learning techniques to process relevant health parameters and generate predictions. Simple Linear Regression, Multiple Linear Regression, and Ridge Regression were explored during the model development process. Python libraries such as NumPy and Pandas were used for data processing and analysis, while Streamlit was used to develop the interactive application interface.

The project also incorporates Generative AI concepts to make the system more user-friendly by providing understandable information based on prediction results. The integration of machine learning with Generative AI and Agentic AI concepts demonstrates how intelligent technologies can be combined to develop practical applications for real-world healthcare problems.

The development of Health Guardian AI provided practical experience in data preprocessing, model development, evaluation, AI integration, application development, testing, debugging, and deployment. The internship also helped strengthen technical and problem-solving skills while providing exposure to modern AI technologies and their practical applications.

Overall, Health Guardian AI demonstrates the application of Artificial Intelligence and Machine Learning in healthcare through an accessible and interactive platform. The project provides a foundation for future improvements such as enhanced datasets, improved predictive models, personalized recommendations, and additional intelligent healthcare features. The internship experience helped bridge the gap between academic concepts and practical AI application development.

INTRODUCTION

HealthGuardian AI is an Agentic AI-powered Personal Health Operating System designed to integrate health information management with AI-assisted guidance. The system combines Artificial Intelligence, Machine Learning, and document processing technologies to provide features such as symptom analysis, medication interaction checks, nutrition tracking, fitness guidance, medical report interpretation, and health-risk prediction.

The application follows a two-component architecture consisting of a Streamlit frontend and a Django backend. The backend manages user profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories using an SQLite database. The frontend provides an interactive interface through which users can access the application's health-related features.

A key feature of HealthGuardian AI is its multi-agent architecture implemented using LangGraph. It coordinates specialized agents, including Planner, Memory, Symptom, Medication, Nutrition, Fitness, Health Risk, and Medical Report agents. The Planner Agent uses the Gemini API to determine the appropriate processing path for a user query, while the Memory Agent retrieves relevant information from the FAISS vector database to provide contextual responses.

The system also incorporates Scikit-learn machine learning models for selected health-related predictions. Simple and Multiple Linear Regression models estimate systolic blood pressure and calories burned, respectively, while Random Forest classifiers generate cardiovascular and diabetes risk estimates. Additionally, PyMuPDF and EasyOCR support the extraction of information from uploaded medical reports.

The primary objective of this project is to demonstrate how AI agents, machine learning models, databases, and document-processing tools can work together within a unified health information platform. The application is intended to support health-information management and provide AI-assisted guidance; its predictions and responses should not replace professional medical advice or diagnosis.

RELATED WORK / TECHNOLOGIES USED

Healthcare applications increasingly use digital technologies to manage health information, maintain medical records, monitor health parameters, and provide accessible health guidance. Traditional health management systems primarily focus on storing patient details, tracking medications, and organizing medical information. Modern AI-powered applications extend these capabilities by incorporating machine learning, natural language processing, medical document processing, and intelligent information retrieval.

Artificial Intelligence enables applications to process user queries and generate relevant responses, while Machine Learning models analyze input parameters to produce predictions. Document-processing technologies extract information from medical reports, and database systems maintain structured health records. Multi-agent architectures further improve task organization by assigning different responsibilities to specialized agents coordinated through a common workflow.

HealthGuardian AI – An Agentic AI-Powered Personal Health Operating System integrates these technologies into a unified application. It uses Streamlit for the frontend, Django for backend development, and SQLite for structured data storage. Its features include symptom analysis, medication interaction checking, nutrition monitoring, fitness guidance, medical report interpretation, and health-risk estimation.

A major feature of the application is its cyclical multi-agent network implemented using LangGraph. The network consists of eight specialized agents responsible for query planning, contextual memory, symptom analysis, medication checks, nutrition analysis, fitness guidance, health-risk estimation, and medical report interpretation. The application also integrates the Google Gemini API for language-model functionality, FAISS for contextual retrieval, and Scikit-learn for machine learning predictions. PyMuPDF and EasyOCR support medical document extraction.

The integration of these technologies demonstrates how frontend development, backend services, database management, machine learning, and AI-agent workflows can be combined to develop an intelligent health information application.

Development Approach

HealthGuardian AI follows a modular architecture consisting of a Streamlit frontend and a Django backend. The frontend provides an interactive interface for accessing health-related features, while the backend manages application logic, user information, and data operations.

Django REST Framework provides API endpoints for communication between the frontend and backend. These endpoints support CRUD operations and provide access to machine learning prediction services. SQLite stores user profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories.

The AI functionality is implemented using a cyclical LangGraph workflow. The Planner Agent uses the Gemini API to determine an appropriate processing path for user queries. The Memory Agent retrieves relevant contextual information using FAISS, including information from previous discussions and available health records. The other specialized agents handle symptoms, medication interactions, nutrition, fitness, medical reports, and health-risk estimates.

The machine learning component consists of two regression models and two Random Forest classifiers developed using Scikit-learn. The Simple Linear Regression model estimates systolic blood pressure based on age, while the Multiple Linear Regression model estimates calories burned using exercise duration, heart rate, and weight. The Random Forest classifiers generate cardiovascular and diabetes risk estimates. The trained models are saved as binary files and accessed through Django prediction APIs.

Medical document processing is performed using PyMuPDF and EasyOCR. PyMuPDF extracts text from PDF documents, while EasyOCR recognizes text from supported medical report images. The extracted information can then be processed by the Medical Report Agent.

This development approach separates the application's major functions into individual components and integrates them through APIs and coordinated AI workflows.

Technologies and Tools Used

The project uses a combination of frontend, backend, database, artificial intelligence, machine learning, and document-processing technologies.

Frontend and Backend Technologies

Streamlit: Streamlit is used to develop the frontend interface. It provides an interactive environment for accessing application features and interacting with the AI agents. The interface uses a medical-themed dark design with glassmorphism styling and visual indicators.

Django and Django REST Framework: Django manages backend functionality, including user authentication and health-related records. Django REST Framework provides REST API endpoints for data operations and machine learning predictions.

Database and Artificial Intelligence Technologies

SQLite: SQLite stores structured information such as user profiles, health vitals, medication reminders, nutrition records, exercise logs, goals, and chat histories.

LangGraph: LangGraph coordinates the eight specialized AI agents through a cyclical workflow, enabling different types of user queries to be processed by appropriate components.

Google Gemini API: The Gemini API provides language-model functionality and supports the Planner Agent in determining the processing path for user queries. The langchain-google-genai package supports its integration with the application's AI components.

FAISS and Retrieval-Augmented Generation: FAISS provides vector-based retrieval for contextual memory. Retrieval-Augmented Generation (RAG) incorporates relevant retrieved information into the AI workflow to support context-aware responses.

Machine Learning and Document Processing

Scikit-learn: Scikit-learn is used to implement the application's machine learning models. These include Simple Linear Regression for systolic blood pressure estimation, Multiple Linear Regression for calorie-burn estimation, and Random Forest classifiers for cardiovascular and diabetes risk estimation.

Pandas and NumPy: These libraries support data processing and numerical operations associated with the machine learning components.

PyMuPDF and EasyOCR: PyMuPDF extracts text from PDF documents, while EasyOCR recognizes text in supported medical report images. These technologies support medical report digitization and further processing by the Medical Report Agent. PyTorch is included as a supporting technology for the OCR component.

Plotly: Plotly supports the visualization of relevant numerical and health-related information within the application.

Python serves as the primary programming language connecting the frontend, backend, AI agents, machine learning models, and document-processing components. Together, these technologies enable HealthGuardian AI to provide integrated health-information management and AI-assisted guidance through a unified application.

WORK DONE / OBSERVATION / DUTIES PERFORMED

System Architecture of Proposed System

HealthGuardian AI – An Agentic AI-Powered Personal Health Operating System follows a modular architecture consisting of a frontend application, backend server, database, machine learning models, medical document-processing components, and a coordinated multi-agent network. These components work together to manage health-related information and provide AI-assisted guidance through a unified application.

The frontend is developed using Streamlit and provides an interactive interface with a medical-themed dark design, glassmorphism styling, and visual indicators. Users can interact with the application's health-related features through this interface. Plotly is used to present relevant information through visualizations.

The backend is implemented using Django and Django REST Framework. It manages user authentication, profile information, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories. REST API endpoints facilitate communication between the frontend and backend and support data-management operations. The backend also provides APIs for executing machine learning predictions.

SQLite is used to store structured application information. It maintains records required by the different modules of the system. The machine learning component consists of Simple Linear Regression, Multiple Linear Regression, and Random Forest classification models developed using Scikit-learn. These models generate systolic blood pressure estimates, calorie-burn estimates, and cardiovascular and diabetes risk estimates.

Another major component is the LangGraph-based multi-agent network, which coordinates eight specialized agents: Planner, Memory, Symptom, Medication, Nutrition, Fitness, Health Risk, and Medical Report. The Planner Agent uses the Gemini API to determine the appropriate processing path for a query. The Memory Agent retrieves relevant contextual information using FAISS, while the other agents handle their respective health-related tasks.

The system also incorporates PyMuPDF and EasyOCR for medical document processing. PyMuPDF extracts text from PDF documents, while EasyOCR recognizes text in supported medical report images. The extracted information can be processed by the Medical Report Agent for interpretation.

The overall architecture integrates frontend interaction, backend processing, database operations, machine learning predictions, document extraction, and AI-agent coordination into a single application.

System Architecture of HealthGuardian AI

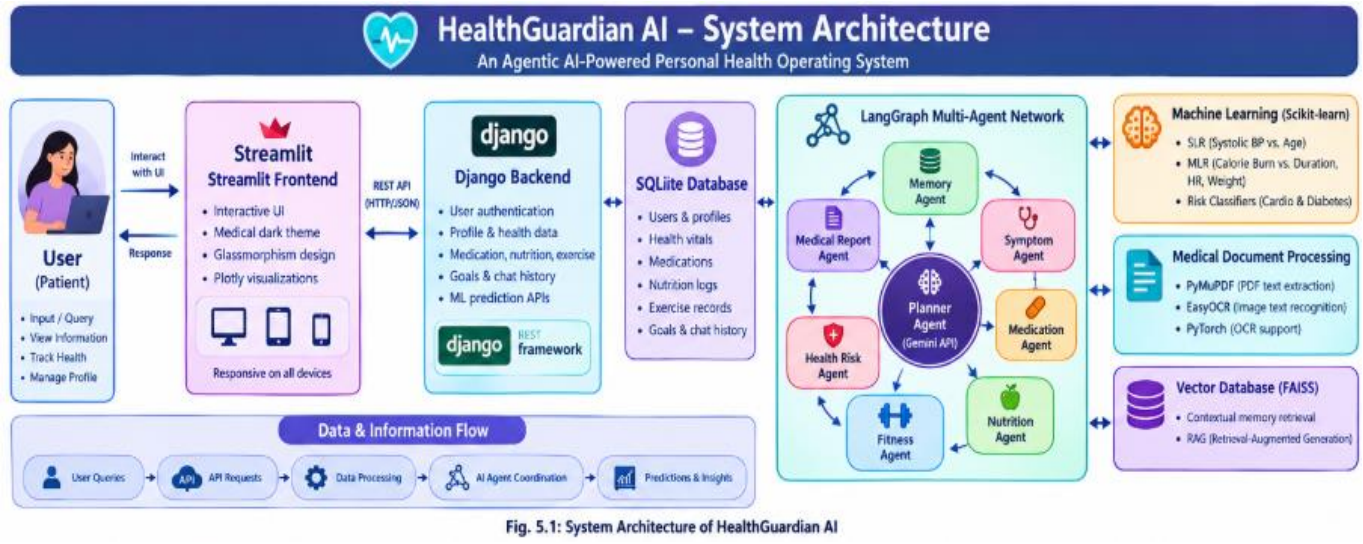


Fig. 5.1: System Architecture of HealthGuardian AI

Data Flow

The data flow of HealthGuardian AI describes how information moves between the user interface, backend services, database, machine learning models, and AI agents. The process begins when a user interacts with the Streamlit frontend to access a health-related feature or submit a query.

When an operation requires stored information, the frontend communicates with the Django backend through REST API endpoints. The backend processes the request and performs the required database operation using SQLite. Information retrieved from the database can then be returned to the frontend for display or made available to the relevant application component.

For AI-assisted queries, the LangGraph workflow coordinates the processing of information through the specialized agents. The Planner Agent evaluates the query using the Gemini API and determines an appropriate processing path. When contextual information is required, the Memory Agent retrieves relevant information through FAISS and incorporates available health-profile details, previous discussions, meal entries, workouts, and vitals into the working context.

Depending on the query, the workflow can involve the Symptom Agent, Medication Agent, Nutrition Agent, Fitness Agent, Health Risk Agent, or Medical Report Agent. Each agent handles its assigned task and contributes to the processing workflow.

For machine learning predictions, the frontend communicates with the Django prediction APIs. The backend loads the appropriate saved Scikit-learn model and processes the supplied input parameters. The Simple Linear Regression endpoint accepts age to estimate systolic blood pressure, while the Multiple Linear Regression endpoint uses exercise duration, heart rate, and weight to estimate calories burned. The risk-prediction endpoint generates cardiovascular and diabetes risk estimates and saves prediction records in the database.

For medical report processing, the application uses PyMuPDF to extract PDF text and EasyOCR to recognize text from supported images. The extracted information can then be passed to the Medical Report Agent for interpretation.

Finally, the processed information or generated response is presented through the frontend. This data flow enables the application's components to work together while maintaining a separation between user interaction, data storage, prediction services, and AI processing.

Data Flow of HealthGuardian AI

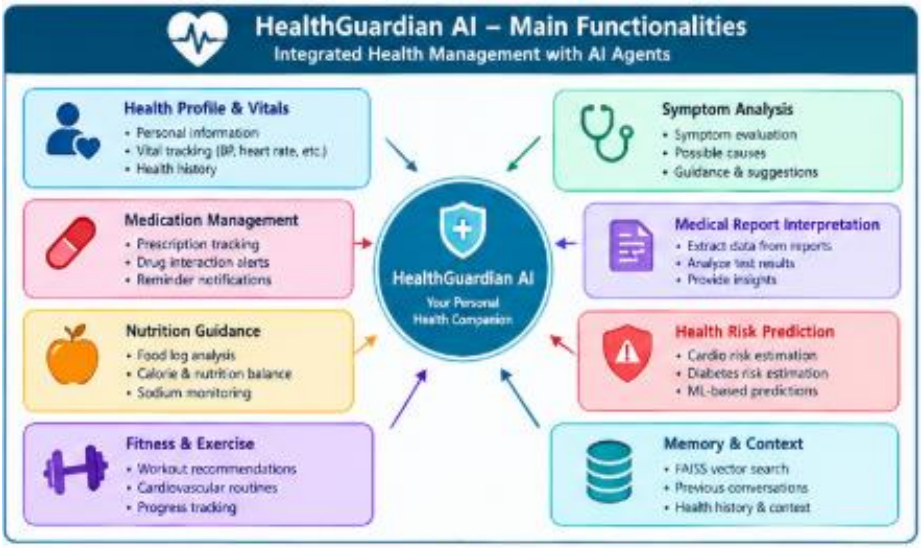


Fig. 5.2: Main Functionalities of HealthGuardian AI

Data Sets

HealthGuardian AI uses structured health records, user-provided information, medical report content, and model-specific input parameters to support its different functionalities. The SQLite database stores application records, while the machine learning models process relevant numerical inputs to generate predictions.

The structured data maintained by the backend includes user profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories. These records provide information required by the application's data-management features and can contribute contextual information to the AI workflow.

The machine learning component uses different input parameters depending on the prediction task. The Simple Linear Regression model uses age as an input to estimate systolic blood pressure. The Multiple Linear Regression model uses exercise duration, heart rate, and weight to estimate calories burned. The Random Forest classifiers generate cardiovascular and diabetes risk estimates using their configured model inputs.

The application also processes information extracted from medical reports. PyMuPDF extracts text from PDF files, while EasyOCR recognizes text in supported medical report images. The

extracted content can be processed by the Medical Report Agent to assist with interpreting laboratory report components.

The Memory Agent uses FAISS-based retrieval to access relevant information from previous discussions and build contextual information using available health profiles, meal entries, workout records, and vitals. This retrieved context supports the AI workflow when relevant information is required.

The README does not specify the complete training datasets, their sizes, or the individual feature columns used by the two risk classifiers. Therefore, these details should only be added if they are confirmed from the actual model-training code.

Medical Data Extraction and Processing

Key Features Extracted for Functionality

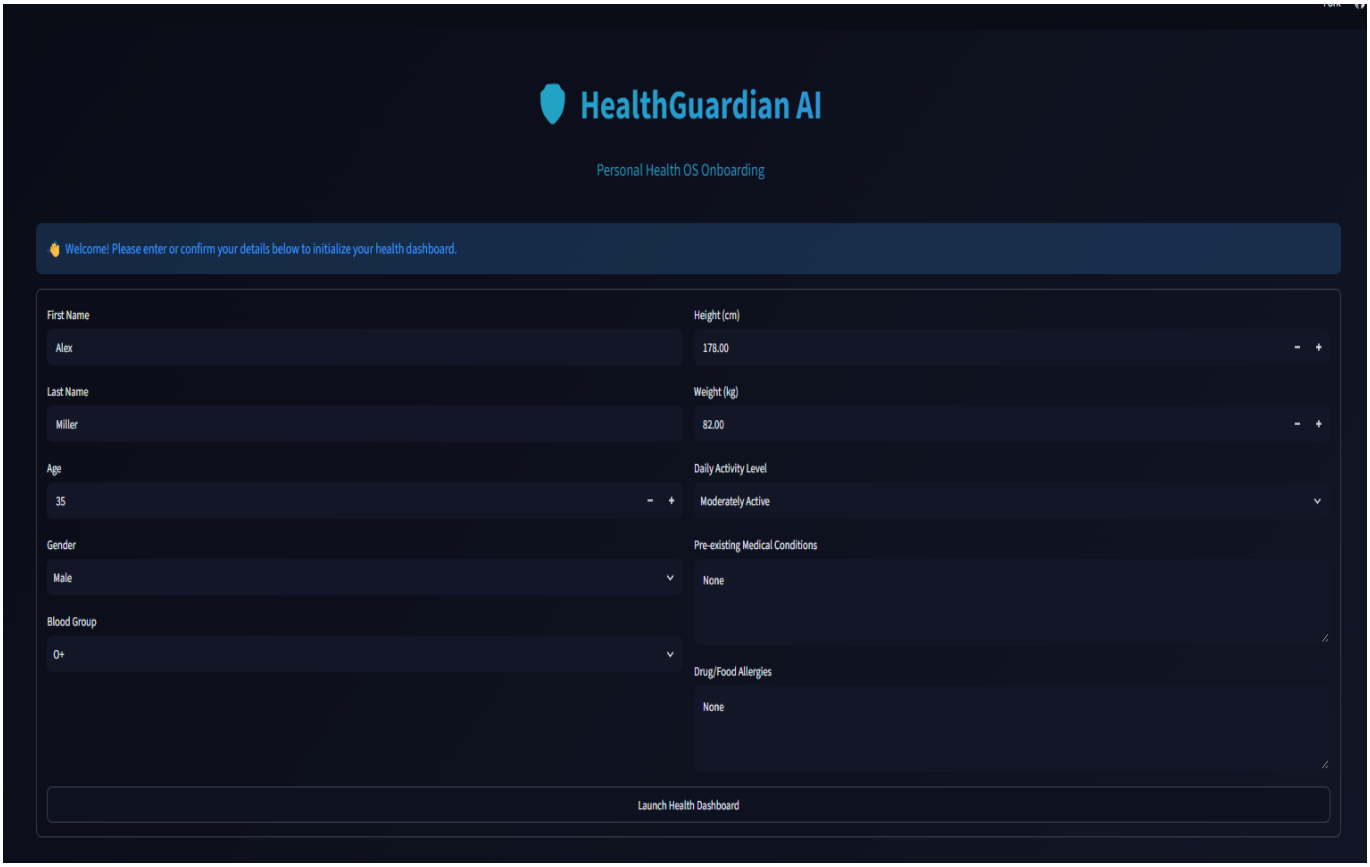
The main information used by HealthGuardian AI can be grouped according to its purpose:

- **Health records:** User profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories.
- **Regression model inputs:** Age for systolic blood pressure estimation, and exercise duration, heart rate, and weight for calorie-burn estimation.
- **Risk prediction:** Input parameters processed by the cardiovascular and diabetes Random Forest classifiers.
- **Medical report information:** Text extracted from PDF documents and supported images for further interpretation.
- **Contextual information:** Relevant previous discussions and available health records retrieved through the FAISS-based Memory Agent.

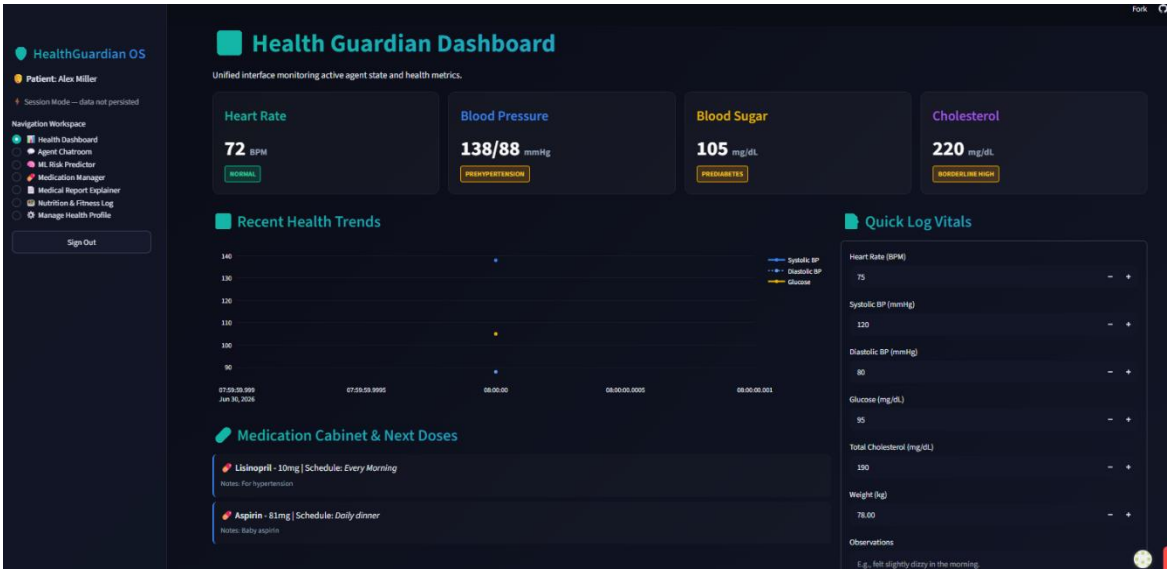
These data components support the application's health-information management, prediction, document-processing, and AI-assisted guidance features. Their integration allows HealthGuardian AI to organize different types of information within a unified application.

Screens

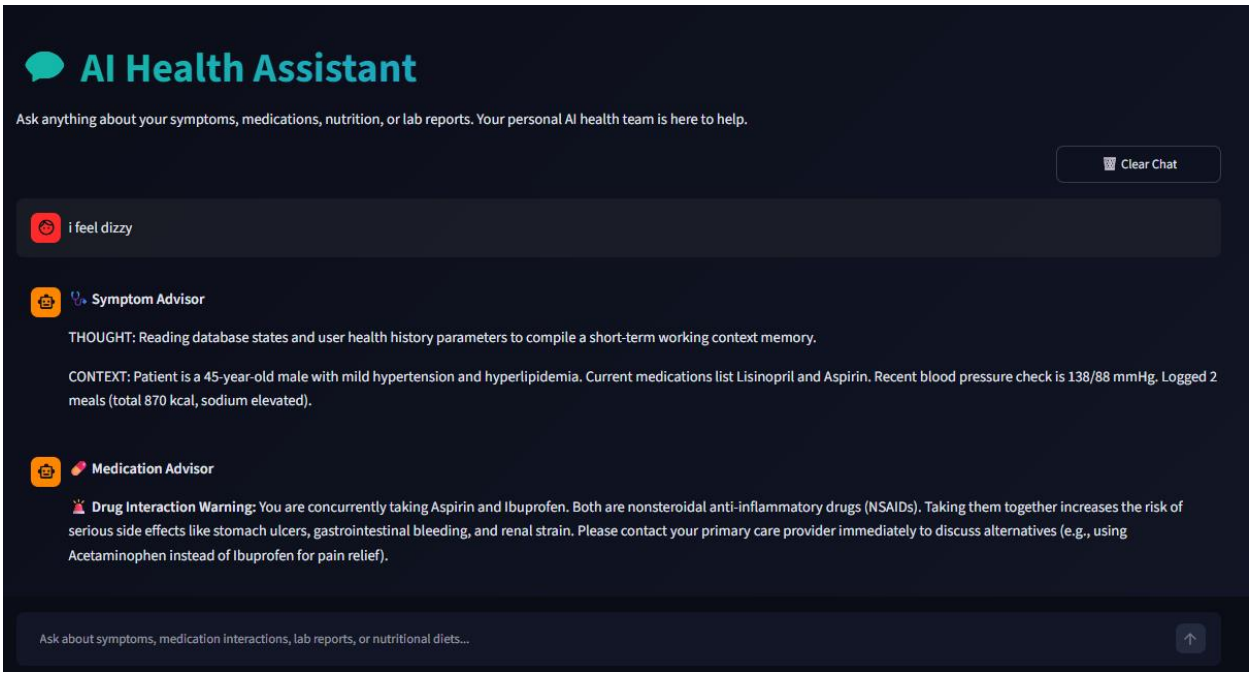
5.4.1 Home Page



5.4.2 Dashboard



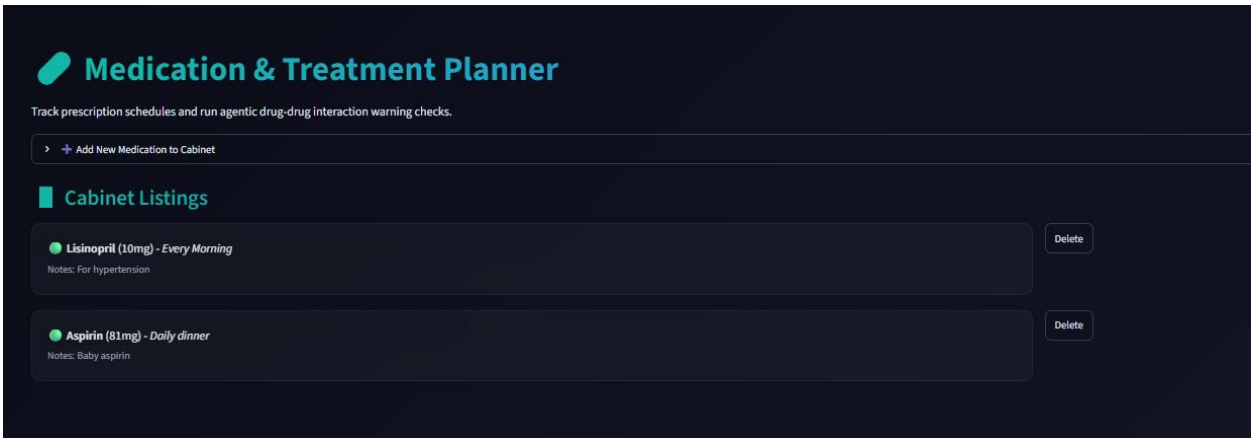
5.4.3 AI Chatbot



5.4.4 Risk Predictor



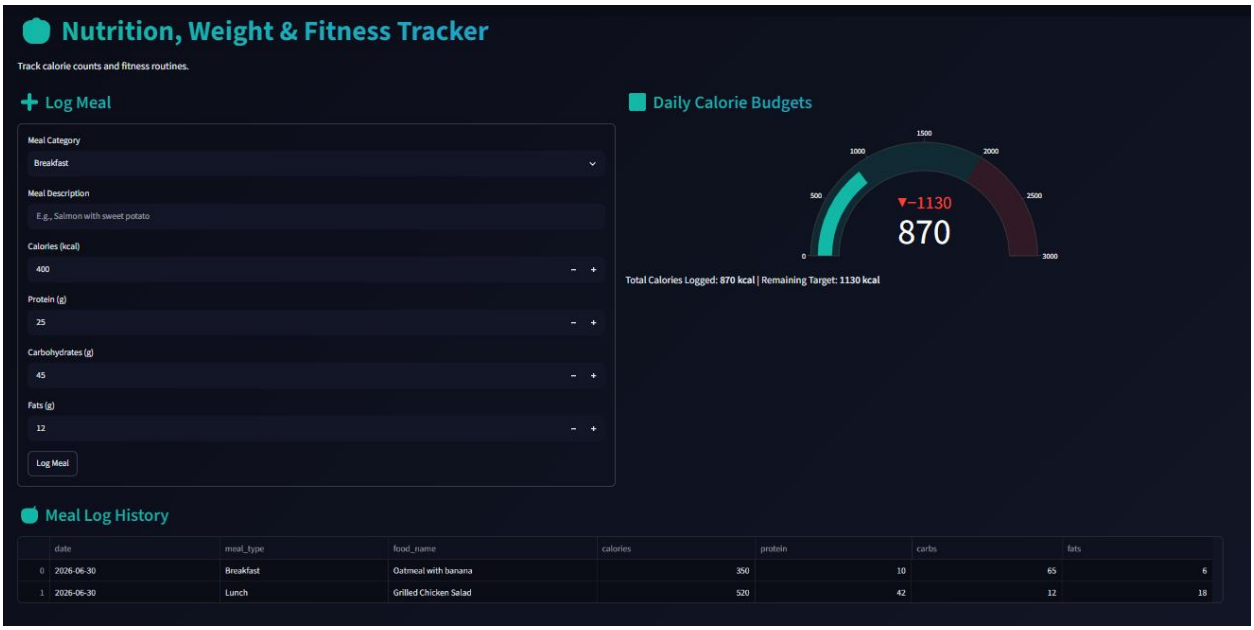
5.4.5 Medication manager



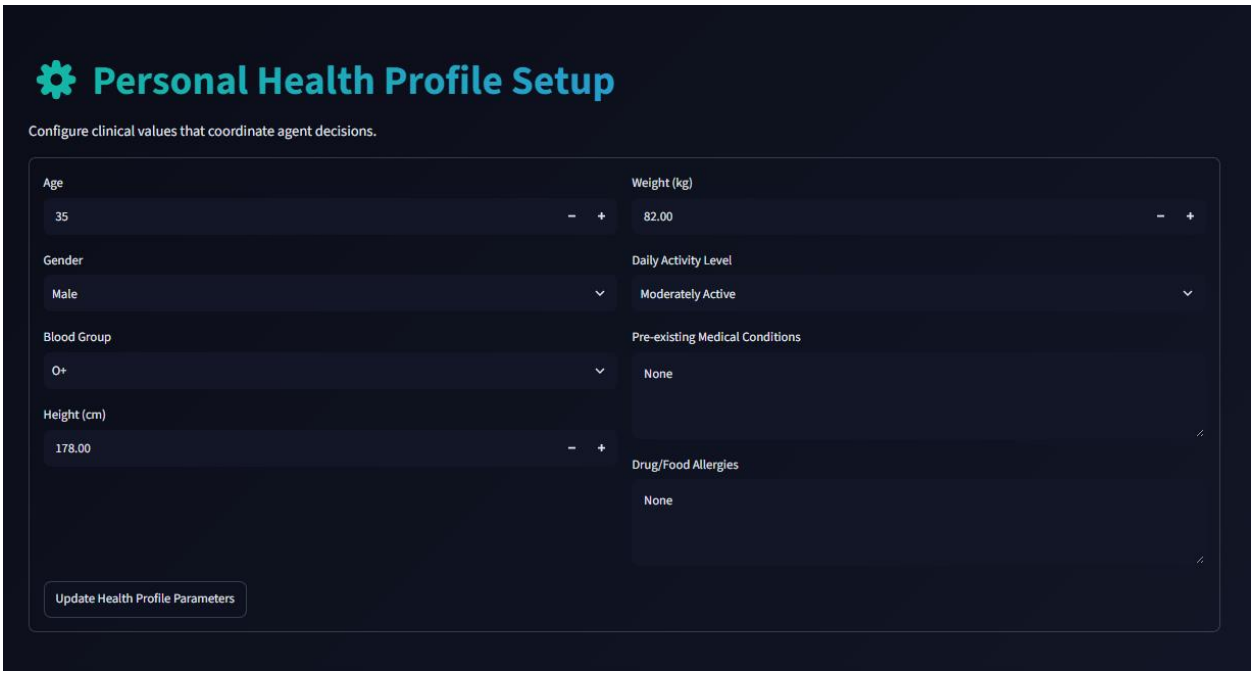
5.4.6 Medical report analyzer



5.4.7 Nutrition and Fitness log



5.4.8 Health Profile



LEARNING AFTER INTERNSHIP

The internship provided valuable practical exposure to Artificial Intelligence, Machine Learning, and Agentic AI development. It helped me understand how theoretical concepts can be applied to develop a practical AI-powered application. Working on **HealthGuardian AI – An Agentic AI-Powered Personal Health Operating System** improved my understanding of frontend development, backend functionality, database management, machine-learning predictions, document processing, and multi-agent coordination.

I gained practical knowledge of Python programming and learned how different Python libraries and frameworks can be integrated to develop an intelligent application. I understood how Streamlit can be used to develop an interactive user interface for displaying health information, collecting user inputs, and presenting results. I also learned how Plotly can be used to visualize health-related information through graphical representations.

The internship improved my understanding of backend development through Django and Django REST Framework. I learned how backend services manage authentication, user profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories. I also gained knowledge of REST APIs and how HTTP requests and responses facilitate communication between the frontend and backend components.

I gained practical understanding of database management through SQLite and learned how structured health information can be stored and retrieved. The project helped me understand how databases support the management of user profiles, health records, medication information, nutrition logs, exercise records, and conversation histories. I also learned about the importance of maintaining organized data to support application functionality.

The internship provided valuable exposure to Agentic AI through LangGraph and the integration of the Google Gemini API. I learned how specialized AI agents can be coordinated to handle different types of user queries. The Planner Agent determines an appropriate execution path, while the Symptom, Medication, Nutrition, Fitness, Medical Report, Health Risk, and Memory Agents perform their respective tasks. This helped me understand how multiple AI agents can work together to provide context-aware responses.

I also improved my understanding of Machine Learning using Scikit-learn, Pandas, and NumPy. The project includes Simple Linear Regression for estimating systolic blood pressure based on age, Multiple Linear Regression for estimating calories burned based on workout duration, heart rate, and weight, and Random Forest classifiers for cardiovascular and diabetes risk estimation. I learned how trained models can be saved and integrated into backend prediction APIs for generating predictions.

Another important area of learning was medical document processing. I gained knowledge of how PyMuPDF can extract text from PDF documents and how EasyOCR can recognize text in scanned medical reports and images. I also learned about FAISS and Retrieval-Augmented Generation (RAG), which support contextual information retrieval using previous conversations and relevant health information.

Apart from technical skills, the internship improved my problem-solving, debugging, integration, and documentation skills. Working on HealthGuardian AI helped me understand how frontend interfaces, backend services, databases, machine-learning models, document-processing tools, and AI agents can be combined into a unified application.

Overall, the internship strengthened my technical knowledge and provided practical experience in developing an AI-powered software system. It helped me apply concepts learned during my academic studies to a practical project and improved my understanding of how Artificial Intelligence, Machine Learning, and Agentic AI can be integrated into software development.

SUMMARY / CONCLUSION

HealthGuardian AI – An Agentic AI-Powered Personal Health Operating System was developed as an AI-powered application designed to organize health-related information and provide personalized health guidance through specialized AI agents. The system integrates health data management, symptom analysis, medication interaction checks, nutrition guidance, fitness recommendations, medical report interpretation, and health-risk estimation within a unified platform.

The application combines modern frontend and backend technologies to provide an integrated software solution. Streamlit is used to develop the interactive user interface, while Django and Django REST Framework handle backend operations and REST API communication. SQLite is used to store structured information, including user profiles, health vitals, medication reminders, nutrition logs, exercise records, goals, and chat histories.

A key feature of the system is its multi-agent architecture implemented using LangGraph and integrated with the Google Gemini API. The Planner Agent selects an appropriate execution path based on the user's query, while specialized agents handle different health-related tasks. The Memory Agent uses FAISS-based retrieval to access relevant contextual information, supporting responses that consider available health profiles and previous interactions.

The system also incorporates Machine Learning models developed using Scikit-learn. Simple Linear Regression and Multiple Linear Regression are used for selected health-related numerical predictions, while Random Forest classifiers estimate cardiovascular and diabetes risk. These models are integrated with Django prediction APIs, allowing the application to process inputs and return prediction results.

Medical document processing is another important component of HealthGuardian AI. PyMuPDF supports text extraction from PDF documents, while EasyOCR is used for text recognition from scanned medical reports and images. The extracted information can then be processed by the Medical Report Agent to provide explanations of report components and relevant health insights.

The development of HealthGuardian AI provided practical experience in Python programming, frontend development, backend services, database management, REST API integration, machine learning, document processing, vector databases, and multi-agent AI systems. The project demonstrated how these technologies can be integrated to develop an application that combines structured health-data management with AI-powered information processing.

Overall, HealthGuardian AI demonstrates the practical application of Artificial Intelligence, Machine Learning, and Agentic AI in developing a personal health information platform. It provides a foundation for further improvements in health-data analysis, contextual assistance, medical document interpretation, and personalized wellness support. The application is intended to support access to health information and should not replace professional medical advice, diagnosis, or treatment.

Future Scope

HealthGuardian AI can be enhanced by introducing additional capabilities such as improved health-data visualization, expanded medical document formats, multilingual interaction, and more advanced contextual retrieval. The application could also incorporate integrations with wearable devices and health-monitoring systems to support the collection of health vitals, subject to appropriate permissions and data-protection requirements.

Future versions could improve the evaluation of machine-learning models by using suitable validated datasets, measuring model performance, and refining prediction methods. The multi-agent workflow could also be enhanced through improved error handling, agent evaluation, and more comprehensive testing of interactions between specialized agents.

Additional improvements may include secure deployment configurations, stronger privacy controls, enhanced authentication, data backup mechanisms, and more detailed health-history summaries. These enhancements could help improve the reliability, usability, and scalability of the application while maintaining appropriate safeguards for sensitive health information.

BIBLIOGRAPHY

1. Django Software Foundation, *Django Documentation*. Official documentation for Django web application development.
2. Django REST Framework, *Django REST Framework Documentation*. Official documentation for developing REST APIs.
3. Streamlit, *Streamlit Documentation*. Official documentation for building interactive Python applications.
4. Python Software Foundation, *Python Documentation*. Official documentation for Python programming.
5. Google, *Gemini API Documentation*. Documentation for integrating Google's generative AI models into applications.
6. LangChain, *LangGraph Documentation*. Documentation for building and coordinating stateful, multi-agent workflows.
7. Scikit-learn Developers, *Scikit-learn User Guide*. Documentation for machine-learning algorithms, model training, and prediction.
8. SQLite, *SQLite Documentation*. Official documentation for the SQLite relational database.
9. Facebook AI Similarity Search (FAISS), *FAISS Documentation*. Documentation for efficient similarity search and vector indexing.
10. PyMuPDF, *PyMuPDF Documentation*. Documentation for PDF processing and text extraction.
11. JaidedAI, *EasyOCR Documentation*. Documentation for optical character recognition and text extraction from images.
12. PyTorch Contributors, *PyTorch Documentation*. Documentation for machine-learning and deep-learning functionality used by supporting libraries.
13. Plotly Technologies Inc., *Plotly Python Graphing Library Documentation*. Documentation for creating interactive graphs and visualizations.
14. Google, *Google Gen AI Python SDK Documentation*. Documentation for programmatic access to Google's generative AI services.
15. HealthGuardian AI, *Project README and Technical Documentation*. Project description, architecture, technologies, machine-learning models, AI agents, and API endpoints.

PROGRAM OUTCOMES (POs)

Program Outcomes (POs) describe what students are expected to know and be able to do by the time of graduation to accomplish Program Educational Objectives (PEOs). The Program Outcomes for CSE-Artificial Intelligence and Machine Learning graduates are:
Engineering Graduates would be able to:

PROGRAM OUTCOMES

1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

3. Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

4. Conduct investigations of complex problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

5. Engineering tool usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

6. The engineer and the world: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

8. Individual and Collaborative team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

9. Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

10. Project management and finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

11. Life-long learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES:

PSO 1: Demonstrate competency in Programming and problem-solving skills and apply those skills in solving computing problems

PSO 2: Select appropriate programming languages, Data structures and algorithms in combination with modern technologies and apply them in developing innovative solutions

PSO 3: Demonstrate adequate knowledge in the concepts and techniques of artificial intelligence and machine learning, apply them in developing intelligent systems to solve real world problems.

Table1: Correlation between the Internship Project and Program Outcomes (POs), Program Specific Outcomes (PSOs).

Roll No	Name	Title

Table2: Project correlation with appropriate POs/PSOs (Please specify level of correlation H, M, L against POs/PSOs).

H	High	M	Medium	L	Low

Internship Project correlation with appropriate POs/PSOs

PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3

Table3: Nature of Internship work (Please tick ✓ appropriate for your Project)

Roll Number	Title	Nature of Project			
		Product	Application	Research	Others (Please Specify)

Table4: Nature of Internship Project work (Please tick ✓ appropriate for your Project)

Roll Number	AI	ML/DL	Web Applications/Mern Stack	Network Security	IoT	Others (Please Specify)